GitHub Copilot Dev Days

GitHub Copilot CLI & Agentic Coding

Debjoty Mitra

Co-founder & CTO, Arbor AI Studio

ChatTodoListService.tsx × ChatMessage.tsx ×

```typescript
import { Todo } from '../types/todo.types';
import { StorageService } from './StorageService';

export class ChatTodoListService {
  private storageService: StorageService;
  private eventListeners: ((todos: Todo[]) => void)[] = [];

  constructor() {
    this.storageService = new StorageService();
  }

  // Existing methods (read/write)
  getTodos(): Todo[] {
    return this.storageService.get('todos') || [];
  }

  setTodos(todos: Todo[]): void {
    this.storageService.set('todos', todos);
    this.emitChange(todos);
  }

  // NEW METHOD — Step 1 of the plan
  updateTodo(updatedTodo: Todo): void {
    const todos = this.getTodos();
    const index = todos.findIndex(todo => todo.id === updatedTodo.id);

    if (index !== -1) {
      todos[index] = { ...todos[index], ...updatedTodo };
      this.storageService.set('todos', todos);
      this.emitChange(todos);
    }
  }

```

CHAT

allow editing the todo title in the chat tool list

Add interactive checkbox toggling and inline editing to the chat todo list widget for manual task updates alongside AI management.

Steps:

1. Implement updateTodo() in ChatTodoListService for individual updates
2. Add click handlers to cycle status: not-started → in-progress → completed
3. Add inline title editing via double-click with save/cancel actions
4. Extend ManageTodoListTool with update operation for AI modifications
5. Update tests for interactions, status toggling, and editing flows

Open Questions:

1. **Conflict resolution?** (AI overwrites / User locks / Merge with timestamps)
2. **User add/delete todos?** (UI buttons / AI-only / Context menu)
3. **Description editing?** (Inline / Modal / Title-only)

PROCEED FROM PLAN

Start Implementation | Open Plan in Editor

@agents, /comments

Plan ⌄ Auto ⌄

GitHub Copilot CLI – your AI terminal companion

Many modes. Many use cases.

`copilot`

User runs CLI locally on their machine with full interactive UI and prompts

Interactive

`copilot -p`

Non-interactive execution for automation pipelines and CI/CD workflows

Headless

`copilot --server`

Exposes JSON-RPC port for external clients to drive CLI functionality remotely

Server

Installation

Windows (WinGet)

```
winget install
GitHub.Copilot
```

macOS / Linux (Homebrew)

```
brew install copilot-cli
```

Any OS (npm, Node.js ≥ 22)

```
npm install –g
@github/copilot
```

GitHub Copilot CLI

@ to mention files

- Provide Copilot context by @-mentioning files directly in your prompt
- @-mention multiple files at once, including images
- Alternatively drag and drop or paste the file or image to provide context

ALT+V / CMD+V to paste images into CLI

```
~
› [copilot-image-ca5438.png]
shift+tab cycle mode · ctrl+x → / run command
```

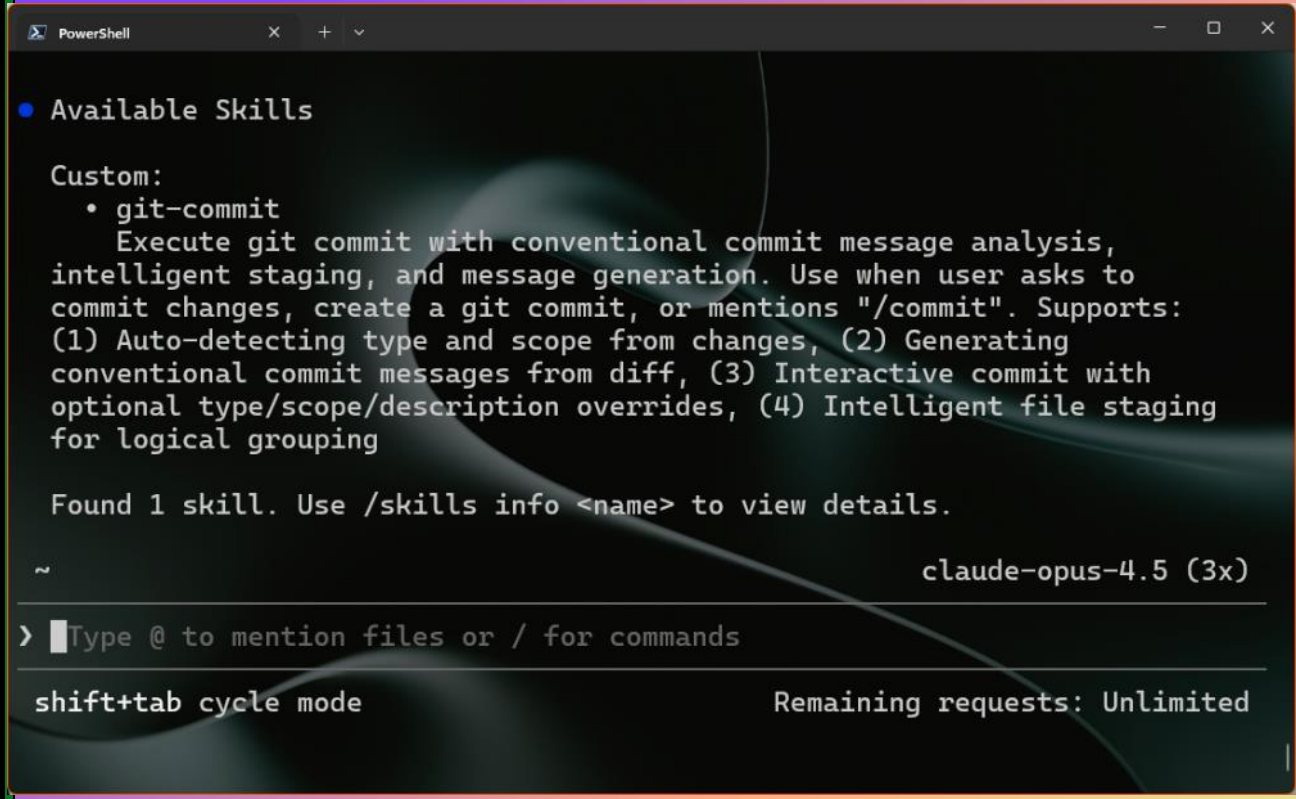
Skills & Plugins

Reusable procedures to extend Copilot's capabilities for repeatable tasks

Add new skills with `/skills add`

Can be project-specific or global in your .copilot/skills or .agents/skills folder

Add new plugins with `/plugin`



Powerful
Built-in Agents

The built-in agents are designed to support a wide range of needs with performance and generalization in mind.

General

Full-featured agent using all CLI tools for complex multi-step tasks.

Research

Run deep research investigation using GitHub search and web sources

Plan

Create a full implementation plan before coding by specify requirements and answering questions

Autopilot

Works through a task without waiting for your input after each step autonomously until it determines the task is complete.

Personalized
Custom Agents

Create an .agent.md file in .github/agents/ or ~/.github/agents/ to get started.

```
---
name: Display Name
description: What the agent does (required)
tools: ["*"]  # Optional - defaults to all tools
mcp-servers:  # Optional
  server-name:
    command: npx
    args: ["@some/mcp-server"]
---
# Agent Instructions
```

Frontmatter Fields

Field	Required	Description
name	No	Display name (defaults to filename)
description	Yes	Describes when to invoke the agent
tools	No	Array of allowed tools (default: all)
mcp-servers	No	MCP server configuration
infer	No	Whether agent can be auto-invoked (default: true)

* More details on our Custom Agents docs page.

Slash commands

`/agent`

Switch to a custom agent

`/model`

View all available models and choose which to use

`/mcp`

Manage your MCP servers

`/yolo`

Enable all permissions

`/diff`

Review changes made in directory

`/skills`

Manage & call skills

`/plan`

Plan out an implementation before writing any code

`/cwd or /cd`

Change working directory mid-session

`/reset-allowed-tools`

Reset Copilot’s tool permissions

`/context`

Info on token usage & context window

`/review`

Run a code review to analyze changes

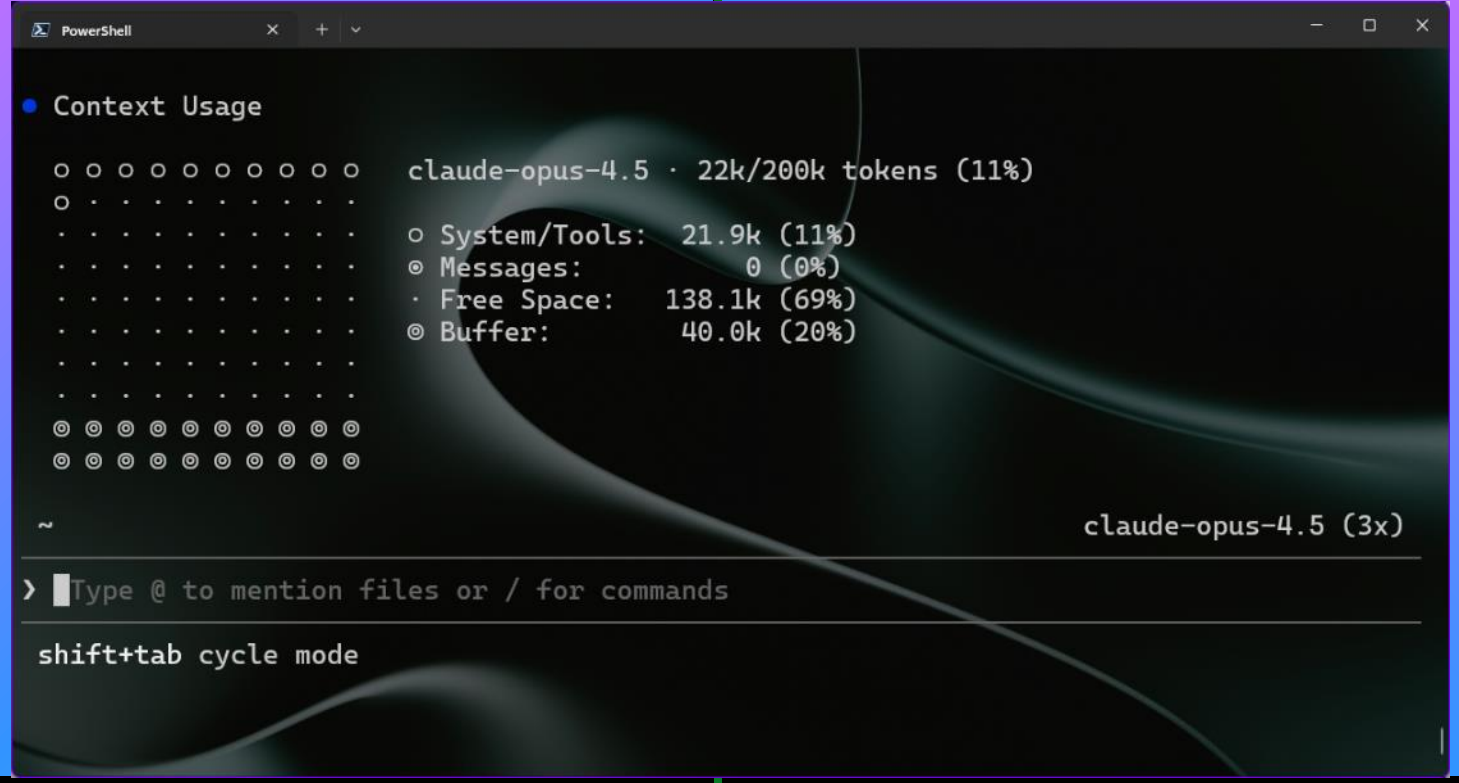
`/plugin`

Manage and install plugins

GitHub Copilot CLI

Context

Infinite Sessions (a CLI exclusive) means you likely never have to worry about these, but when you want to manage your context more directly, you can:



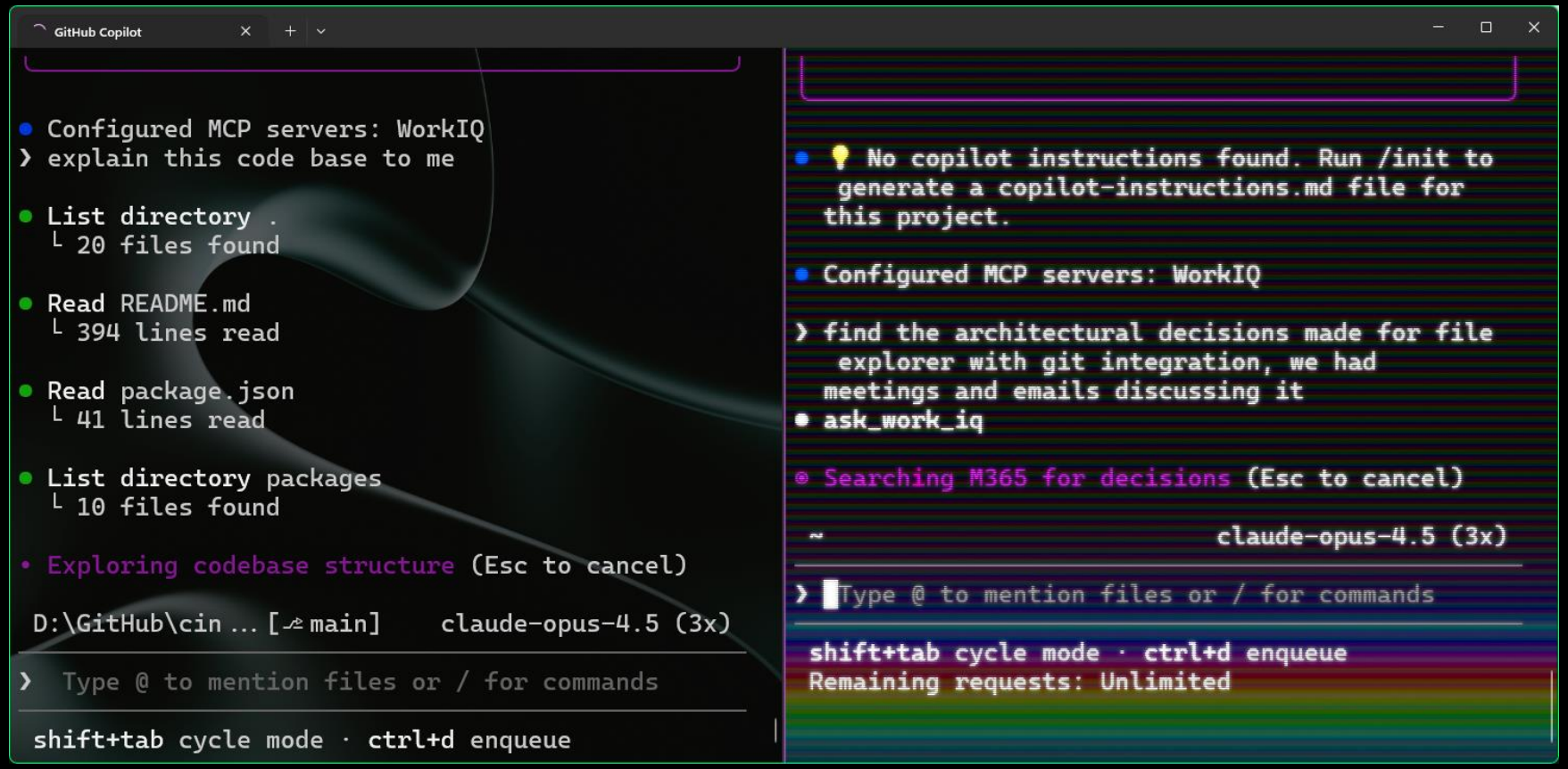
`/context`

Visualize what is in your context window.

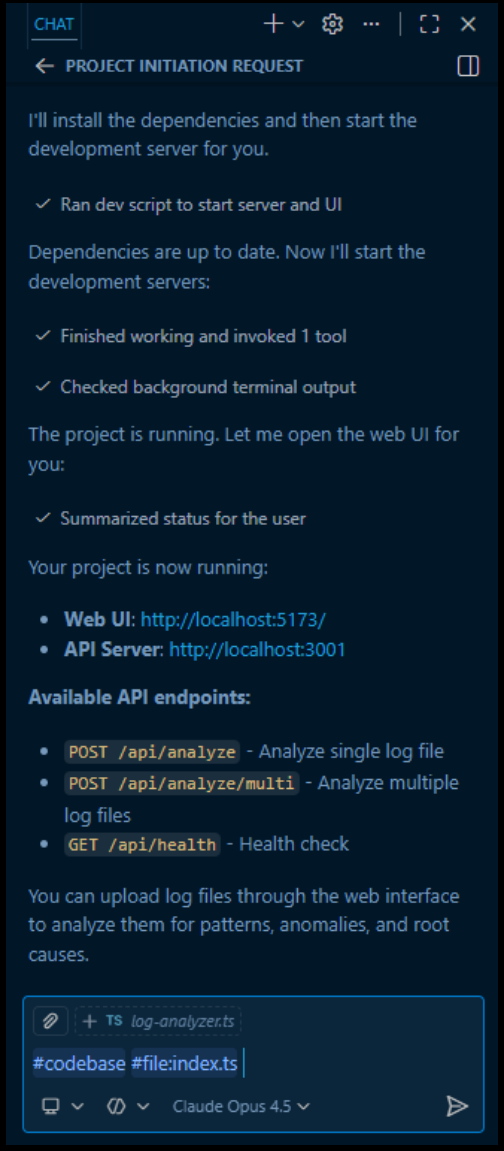
`/compact`

Summarizes current context and compacts to smaller size.

Parallel Copilot



Two projects in terminal split panes



VS Code agent chat

GitHub Copilot Dev Days Sweepstake

All completed surveys are eligible for our sweepstakes to win a GitHub Copilot Pro+ one-year subscription.

https://aka.ms/githubcopilotdevdays/sweepstakes

Thank you

gh.io/copilot